

MediQ 2.1.4 - Complete Repository Onboarding

Structure, configuration, requests, storage and operation

A guide for a reader opening MediQ for the first time.

Snapshot: October 8, 2026. Source branch: `main`. Source commit: `73f5f74befbb1a7c6726ca4e9401814a4311016b`. This commit restores the completed 2.1.4 source after the 2.1.5 merge. The Git history contains that merge and its revert; the current file tree matches `v2.1.4`.

Scope: Every MediQ-owned tracked file, every environment-file role and variable, every project folder, and the local `hidden/data/build/test` folders present in this checkout. Third-party implementation files, compressed Git objects, generated bundles and private records are grouped explicitly. No actual credentials, passwords, OTPs, session tokens, email addresses from local data, or individual database records are reproduced.

Reading order: Read chapters 1-4 to understand the system; chapters 5-10 to find files; chapters 11-14 to follow requests and operate it; appendices to check coverage. The PDF and this README carry the same explanatory content. The existing root README is preserved.

This is a description of the current implementation, not a deployment, a promise that vendor integrations work, or a claim that old planning documents are current specifications.

Table of contents

1. Architecture overview

3

2. What is real, fictional, temporary, and persistent

4

3. Repository and folder map

5

4. Environment files and every variable

7

5. Browser files: boot, pages and reusable components

9

6. Browser state, services, shared logic and translations

12

7. Server files and database actions

15

8. Root configuration, static assets, scripts, tests, deployment and existing documents

17

9. Dependencies, installation and grouped external files

22

10. Hidden folders, local files and generated output

27

11. End-to-end request and state walkthroughs

29

12. Every npm command, build path and test boundary

32

13. Database tables, keys and API reference

34

14. Working safely in this repository and finding the right file

38

Appendix A. Exact tracked-file coverage and grouped exclusions

40

Appendix B. Folder coverage ledger

44

Appendix C. Glossary and source navigation

46

1. Architecture overview

MediQ is a healthcare visit-planning prototype. A visitor can compare fictional clinics, see total-visit timing examples and confidence, save a plan, and submit operational timing reports. Verified accounts can keep private settings and saved care across devices. The core idea is planning an entire visit, not just the waiting-room segment.

The project is one repository containing a browser application and a separate server application. Locally they are two running processes. In a hosted configuration the server can serve the built browser files and the API from one address.

```

Browser at localhost:4173
index.html -> src/main.tsx -> App.tsx
  AppProvider: session, saves, plans, language, reports
  AppRoutes -> page -> shared components
                    |
                    clinicService / api
                    |
Vite proxy: /api -> localhost:3001
                    |
server/index.ts -> application.ts -> auth / validation
                                   |           |
                                   SQLite      jobs / estimator
                                   |
                                   .data/development/mediq.sqlite
  
```

Alternative browser-only mode:

```

page -> mockClinicService -> src/data/mockData.ts
Guest changes remain in browser memory; no account API.
  
```

Built/hosted mode:

```

npm run build:production -> dist/
server/index.ts with SERVE_STATIC=true serves dist/ + /api
  
```

React renders the interface. TypeScript checks contracts. Vite runs the development server and bundles browser code. Node runs the API. SQLite persists relational data in a file using Node's built-in `node:sqlite`. Better Auth manages passwords, verification and sessions. Zod validates incoming data. There is no Express application, separate Python backend, paid AI search service, or external map SDK in the current source.

A **component** renders reusable UI; a **page** represents a route; a **service** reads or writes application data; a **context** shares browser state; a **migration** defines a database schema change; a **build** turns source into deployable files. A **route** is a URL path, while an **endpoint** is an API operation at a URL plus HTTP method.

Key decisions and reasons:

- Keep data access behind `ClinicDataService` because pages should use the same interface in demo and backend modes.
- Derive private ownership from a server-verified session because a browser-provided user ID is not trustworthy.
- Use one SQLite file per environment because local persistence is simple and environment mixing can corrupt expectations.
- Store clinics as JSON plus related tables because the rich UI object can be returned directly while relationships remain explicit.
- Validate on the server as well as in forms because callers can bypass the browser.
- Keep unsupported integrations explicitly unavailable because a button or fixture is not a real reservation or live data feed.

- Exclude administration from public builds and deny it on the production API because hiding navigation alone is insufficient.

A single file-backed database favors a small pilot and a single server instance. It is not the current architecture for multiple independent API replicas or guaranteed uninterrupted deployments.

2. What is real, fictional, temporary, and persistent

The word "demo" has two different meanings in this project. Fictional **clinic data** can be served through a real backend. Browser-only **demo mode** means no backend accounts or durable report writes. A future investor demonstration with a real API needs backend mode plus an intentionally designed fictional-data release; it is not achieved just by selecting browser-only demo mode.

Capability	Current behavior
Clinic/provider directory	Six fictional Durham-area clinic fixtures with fictional people, contact details, availability and reviews
Search/filtering	Real deterministic code, using either local fixtures or clinic documents in SQLite
Backend signup/login/settings/saves	Real local API and durable SQL storage, gated by verified accounts
Development verification email	A private JSON mailbox file; no message reaches an external inbox
Resend email	Implemented adapter; requires real account/key/sender setup and external delivery testing
Report submission in backend mode	Validated, deduplicated, stored and eligible for moderation/estimation
Report submission in browser-only mode	Temporary result added to current AppContext memory
Guest saves/plans/preferences	Memory only; full reload/new tab resets personal changes
Signed-in saves/plans/preferences	Owner-scoped SQLite data restored through /api/me
Accepted guest backend reports	Shared SQL records survive the guest UI resetting
Historical chart	Fixture records, not a chart automatically reconstructed from estimate_history
Map	Stylized HTML/CSS map using fictional x/y positions; not a live map or route service
Distance	Haversine straight-line miles from approximate optional coordinates, not driving time
Travel and buffer	Manually entered plan values; no routing provider is installed
Booking buttons and queue alerts	Simulated UI responses; no reservation or delivered notification
Contact form	Browser-only confirmation; no message sent or stored
Admin metrics	SQL event counters and recent presence, not billing/revenue/confirmed bookings

Capability	Current behavior
Public deployment	Configuration templates exist; this document does not provision a hosted service

The directory's availability and historical examples do not become real when they are moved into a database. Backend estimates distinguish ready evidence from insufficient or stale evidence. When evidence is insufficient, the server retains labeled fictional reference timing in the returned clinic shape; that number must not be interpreted as a live observed estimate.

The first investor demo is a future deployment goal. Current seedDemo refuses production environments. Any future public fictional-data release must address that intentionally, without copying the local development database or exposing internal administration.

3. Repository and folder map

Paths below are relative to the repository root. "Tracked" means part of Git's current file tree. "Local" means present on disk but not committed. "Generated" means a tool can recreate it. A file may be generated and tracked at the same time.

MediQ_git/	
.git/	Local Git history, index, refs, logs and hooks
.data/	Local databases, local mail, tests and rollback archives
.env*	Browser-side environment configuration and examples
.gitignore	Rules for future untracked files
README.md	Existing project overview, retained
README-LOCAL-SETUP.md	Ignored personal setup guide
package*.json	Dependencies, npm scripts and locked versions
index.html	Browser HTML entry
vite.config.ts	Browser build, development port and API proxy
tsconfig*.json	TypeScript compiler scopes
playwright.config.ts	Browser test runner and test servers
vercel.json	Static SPA rewrite template
src/	
main.tsx, App.tsx	React boot and route selection
components/	Reusable presentation and interaction
pages/	Route screens
context/	Shared state and translation infrastructure
locales/	Six named translated JSON catalogs
data/	Fictional fixture source
services/	API, account contracts, search and location adapters
utils/	Navigation, time math and report helpers
styles.css, types.ts	Visual system and domain contracts
server/	
.env*	Server-only settings/examples
*.ts	API, auth, database, jobs and CLI
migrations/	Numbered schema files
public/	Static files copied verbatim into a build
scripts/	Build checks, static preview and test startup
tests/	
backend.test.ts	API/auth/database tests
browser/	Playwright scenarios
deploy/	Staging and production Render templates
docs/	Existing policy, planning and verification documents
onboarding/	This README and its matching PDF
dist/	Generated deployable browser output
assets/	Bundled CSS/JavaScript
node_modules/	Installed external packages and tool caches
playwright-report/	Generated HTML test report
test-results/	Generated screenshots and last-run status

Every first-party subfolder is shown above. Dependency-specific nested folders and hash-named Git object folders share the grouped purposes explained in chapters 9-10 and appendix B. No `.github/`, `.vscode/`, `.agents/`, `.codex/`, `.aws/`, custom root `AGENTS.md`, or CI workflow folder was present in this repository snapshot. Their absence is not evidence that the hosting account has no settings elsewhere.

`src` runs primarily in the browser. `server` runs in Node. Shared types, deterministic search and report helpers are imported into both where useful. These shared modules must avoid browser-only top-level side effects if the server imports them.

3.1 Every browser route

Browser path	Screen and navigation role
/	OnboardingPage: guest entry, signup/sign-in and fictional preview; independent landing layout.
/find and /find?q=...	FindCarePage: search, constraints, explicit sorts, map/list and plan summary.
/clinic/:id	ClinicPage: details and visit-plan dialog for decoded clinic ID.
/saved	SavedPage: private account saves or temporary guest plans/favorites.
/report and /report?clinic=...	ReportPage: detailed report, optionally prefilling a clinic.
/know	KnowPage: About; #why, #journey, #estimates, #reliability, #limitations, #privacy, #access and #safety anchors.
/contact	ContactPage: unsent form demonstration.
/signup	AccountPage signup mode.
/login	AccountPage login mode.
/verify	AccountPage six-digit verification mode.
/reset-password	AccountPage code request/reset mode.
/settings	SettingsPage: signed-in private settings; guest sees sign-in prompt.
/admin	Lazy AdminPage when browser environment allows it, with server role checks; absent in production browser build.
Any other path	AppRoutes translated not-found screen; missing clinic has its own detail fallback.

A URL hash navigates within a page. Query strings initialize selected search/report fields; they are not a server route declaration. The current helper is a small explicit router, so adding a new page means editing `App.tsx` rather than adding a configuration to an external routing library.

4. Environment files and every variable

Environment variables are configuration values supplied outside source code. Examples are safe templates. Actual `.env` files may contain credentials and must remain private. The inspection recorded variable names and roles, never actual secret values.

4.1 File-by-file environment inventory

File	Status and role
<code>.env</code>	Actual local browser settings; ignored. Contains <code>VITE_DATA_MODE</code> and <code>VITE_APP_ENV</code> . Current mode is backend/development.
<code>.env.example</code>	Tracked browser template for development; contains <code>VITE_DATA_MODE</code> and <code>VITE_APP_ENV</code> .
<code>.env.staging.example</code>	Tracked browser template for staging builds; same two keys.
<code>.env.production.example</code>	Tracked browser template for public production builds; same two keys.
<code>server/.env</code>	Actual local API settings; ignored. Contains <code>APP_ENV</code> , <code>APP_ORIGIN</code> , <code>DATABASE_PATH</code> , <code>PORT</code> , <code>HOST</code> , <code>AUTH_SECRET</code> , <code>MAIL_MODE</code> , <code>SERVE_STATIC</code> and <code>TRUST_PROXY</code> . Actual values are not reproduced.
<code>server/.env.example</code>	Tracked development API template, containing the same nine keys. Placeholder <code>AUTH_SECRET</code> is not a production credential.
<code>server/.env.staging.example</code>	Tracked staging API template; adds <code>RESEND_API_KEY</code> , <code>MAIL_FROM</code> and <code>RUN_JOBS</code> to the development key set.
<code>server/.env.production.example</code>	Tracked production API template; same twelve keys as the staging example, with independent production placeholders.

No actual `.env.staging`, `.env.production`, `.env.local`, `mode-local` file, or additional real server environment file was present at inspection. `.example` files are not loaded automatically just because they exist. A user must copy a chosen example or set equivalent hosting variables. Do not overwrite an existing real environment file while following setup instructions.

4.2 Variable reference

Variable	Consumer, purpose, default and important boundary
VITE_DATA_MODE	src/services/api.ts. Only exact backend enables HTTP-backed service; other/unset values select the mock adapter. Public browser value, never a secret. Release script forces backend.
VITE_APP_ENV	vite.config.ts, src/App.tsx, src/services/api.ts. Marks browser development/staging/production and controls internal UI inclusion. Public compiled value. Production and staging Vite modes override a conflicting development value.
APP_ENV	server/config.ts. Server environment: development, staging, production or test; defaults development. Controls validation, DB path default, mail default and admin availability. Separate from VITE_APP_ENV.
APP_ORIGIN	Server config/auth/request handling. Exact browser origin used for trusted origins, cookie security and write-origin checks; defaults the development localhost origin. Deployed environments require HTTPS. It is the site origin, not an arbitrary third-party API URL.
DATABASE_PATH	Server config/database. Path to SQLite file; default .data/<environment>/mediq.sqlite, resolved from working directory. Hosted templates use permanent storage under /var/data. Never share across environments.
PORT	server/index.ts indirectly through config; defaults 3001 for API. Also used separately by scripts/serve-dist.mjs, where default is 4173, and overridden to 43190 by smoke tests. Same name, different process.
HOST	Server listening interface. Defaults loopback 127.0.0.1; hosting templates use 0.0.0.0 so the host can route requests to the process.
AUTH_SECRET	Better Auth signing and application HMACs. At least 32 characters; use an independent random private secret per deployed environment. Local fallback exists in source, but is not acceptable for a deployed service. Never put it in a VITE_ variable.
MAIL_MODE	server/auth.ts. file writes private local messages; resend calls the provider. Defaults file locally and resend for staging/production. File mode is prohibited in deployed environments.
MAIL_DIRECTORY	Optional server setting supported by config, absent from checked-in env templates and current real env file. Default .data/<environment>/mail; overridden for tests. Contains sensitive OTP messages.
RESEND_API_KEY	Private server email API credential. Required in staging/production and used in a bearer authorization header; absent from local file-mail setup.
MAIL_FROM	Verified sender identity/address for Resend. Required in staging/production; configure through provider and host, not browser code.
SERVE_STATIC	Server config/index. Exact true enables serving dist from the API server. Default false locally because Vite serves source separately.
TRUST_PROXY	Server config/index. Exact true trusts the selected forwarded address behind a controlled reverse proxy for anti-abuse bucketing. Default false locally. Configure only with a known proxy boundary.

Variable	Consumer, purpose, default and important boundary
RUN_JOBS	Read directly by server/index.ts. Exact true starts an immediate job run and non-overlapping five-minute schedule. Default off; no automatic jobs just because the API is running locally.
NODE_VERSION	Hosting configuration in both Render YAML files, set to 24. Chooses host runtime; application code does not read it.

The examples and deploy files contain placeholder configuration, not purchased hosting, verified domain ownership or an active Resend account. Shell-provided environment variables also matter: Node's env-file option and Vite's loading rules do not imply that these files override every external setting.

4.3 How loading and builds work

`npm run dev` starts Vite, which loads root environment files for its mode. Browser-facing `VITE_` values become part of bundled client code. Vite also reads mode-specific files when they actually exist. Root `.env` is not the server credential file.

`npm run api` explicitly uses Node's `--env-file-if-exists=server/.env`; the API and database CLI scripts share this loading convention. Staging/production example files are not automatically selected by that command. A host normally injects variables through its private environment settings.

`build:staging` and `build:production` invoke `scripts/build-release.mjs`, which forces `VITE_DATA_MODE=backend` and the chosen `VITE_APP_ENV` in child processes. Changing these browser values after a build requires rebuilding. Changing a server-only value requires restarting the API process. Matching origins and environments across both sides avoids cookie, access and data mismatches.

5. Browser files: boot, pages and reusable components

The catalogs below give each MediQ-owned file its own entry. Read "purpose" and "connections" together: a page does not normally write SQL, and a component does not own a network account session. Imports assemble the system described in chapter 1.

`src/App.tsx` - Provider composition and route dispatch

Wraps the app in `AppProvider`, waits for account restoration, then chooses a page using the current path from `useLocationPath`. The landing route uses its own layout; other pages use `Layout`. It sets translated page titles and skip-link text. `AdminPage` is lazy-imported only for development/staging browser builds. Missing paths and clinics have distinct fallbacks.

`src/components/Brand.tsx` - Reusable inline brand

`LogoMark` draws a small SVG clock/search motif. `Brand` combines it with the rendered MediQ wordmark in an internal navigation `Link` and supports a compact size. It consumes translated accessibility text. The SVG is code-defined rather than loaded from an external image server.

`src/components/ClinicComponents.tsx` - **Shared clinic presentation and exploration**

`formatMode` maps stable visit codes to display labels. `VisitBreakdown` displays stage proportions/times. `ClinicCard` reads saved state and opens `QuickReport` with the chosen mode. `MapView` synchronizes a selected fictional clinic between positioned markers and a side list. `HistoricalChart` toggles typical/morning/afternoon fixture series in an accessible SVG range chart. `RatingStars` renders score text and an accessible label. Search and clinic pages reuse these components so timing/ratings are displayed consistently.

`src/components/Layout.tsx` - **Shared page shell**

Renders desktop/mobile navigation, role-sensitive account/settings/admin entries, `LanguageSelector`, main content, prototype footer, `ProductTour` and the live toast region. Uses `currentPath` for active state and `AppContext` for account/language. Footer copy distinguishes backend fixtures and stored reports from browser-only demo data. `LanguageSelector` changes interface language, not spoken-language search filters.

`src/components/ProductTour.tsx` - **Guided interface tour**

Defines six steps targeting data-tour attributes: search, filters, stages, reliability, saved appointments and reports. Tracks visible target bounds for highlighting, opens mobile filters if needed, and updates positioning on scroll/resize/interval. Handles skip/Escape, context completion, and the `mediq:tour` replay event. Tour completion is in current browser memory in 2.1.4; it is not a durable account onboarding record.

`src/components/QuickReport.tsx` - **Short wait-report dialog**

Uses `Modal` and `initialDraft` to submit either minutes still waiting or a completed duration range for today. A ref lock prevents duplicate concurrent submits; validation runs before `AppContext.submitReport`. Review/accepted and backend/demo confirmations differ. Ongoing reports explicitly remain separate from completed totals. A link opens the detailed `ReportPage` for earlier dates or exact visit times.

`src/components/UI.tsx` - **Common badges, notices, loading and dialog**

`DemoBadge` labels prototype data; `ReliabilityBadge` renders confidence with accessible text; `SafetyNote` and `InfoNote` explain limitations; `PageLoader` gives status feedback. `Modal` portals a native dialog into `document.body`, opens `showModal`, prevents background scrolling, handles Escape/outside clicks and restores previous focus on cleanup. Native modal behavior provides focus containment rather than a hand-built overlay alone.

`src/main.tsx` - **React startup**

Imports global styles and mounts App into `index.html`'s root with `React.createRoot` and `StrictMode`. `StrictMode` can exercise effect setup/cleanup extra times in development; effects therefore need cleanup and safe asynchronous guards. No database is opened by the browser entry.

`src/pages/AccountPage.tsx` - **Signup, login, verification and recovery routes**

One component branches by mode; `PasswordField` implements show/hide, autocomplete and length requirements. Uses `authRequest` for Better Auth endpoint calls. Signup normalizes email and routes to verification; login announces account change and routes to search; verification requires a later sign-in; recovery requests a code then resets password. `pendingEmail` and `resend` cooldown are module memory, not persistent profile storage. Browser-only mode disables actual account actions.

`src/pages/AdminPage.tsx` - **Internal development/staging operations**

A role-gated lazy page has activity, verified-account directory, review queue, clinic corrections, launch checklist and audit tabs. Refreshes every 30 seconds with request-sequence guards, search and 20-row pagination. TaskCard stores status/notes; ReviewCard submits decision/reason; ClinicCorrection edits only schema-approved directory JSON. Server authorization remains mandatory even when the UI hides the page. Public production builds omit this component.

`src/pages/ClinicPage.tsx` - **Clinic detail route `/clinic/:id`**

Loads one clinic through clinicService with cleanup/retry guards, distinguishes not-found from service failure and chooses a supported visit mode. Shows stages, confidence, providers, fictional reviews, hours, history and recent accepted reports, merging/deduplicating current-context submissions. The plan dialog makes an ISO timestamp with manual travel/buffer values and calls saveAppointment. Booking and queue-alert handlers explicitly simulate outcomes. They do not call scheduling or notification providers.

`src/pages/ContactPage.tsx` - **Unsent contact demo route `/contact`**

Displays a category/message form and a local completion flag. Submit prevents normal form navigation and renders an explicit nothing-sent/nothing-saved confirmation. There is no contact API, email delivery, data persistence or CRM integration behind this page.

`src/pages/FindCarePage.tsx` - **Clinic search route `/find`**

Initializes query from `?q=`, keeps SearchFilters in React state and reloads through clinicService on filter changes. Sequence guards ignore stale responses; error state has retry rather than fixture fallback. Recommended sorting keeps service relevance; explicit distance/shortest/reliability sorts reorder matches. Provides quick filters, optional approximate geolocation, list/map selection and the current saved-plan summary. Updating interface language does not change stable filter values.

`src/pages/KnowPage.tsx` - **About and limitations route `/know`**

Explains purpose, patient journey, whole-visit stages and confidence. Exposes anchors used by footer links for privacy, limitations and safety. Directly imports fixtures to illustrate an example even in backend mode; its example is not calculated from live report evidence. Proper names and some fixture prose are not dynamically translated.

`src/pages/OnboardingPage.tsx` - **Landing route `/`**

Offers guest entry and links to real signup/login screens, an illustrated fictional visit timeline and prototype explanation. Uses Brand and LanguageSelector directly instead of Layout. Guest entry navigates to `/find`; it does not create an account or seed a database. The sample preview is explanatory UI, not a live result from the API.

`src/pages/ReportPage.tsx` - **Detailed report route `/report`**

Loads clinic choices, optionally prefills `?clinic=`, and adapts visit mode to the selected clinic. Offers exact times or a known total range, experience choices and an anonymity control. validateReport drives focused errors; submission clears irrelevant timing/range fields before the shared service call. Busy state avoids concurrent submissions. Confirmation differentiates review, accepted and demo behavior; reset makes a fresh draft. The backend does not retain free-text notes/exact timing despite form inputs.

`src/pages/SavedPage.tsx` - **Saved clinics and plans route /saved**

Reads saved state from AppContext, loads distinct referenced clinic IDs in parallel and builds an ID lookup. Displays leave-by, arrival, visit-start and estimated finish through `getVisitPlan`. Remove/toggle actions delegate to context, which selects persistent account writes or temporary guest updates. Distinguishes a sample guest appointment from personal plans and explains that saved visits are not bookings.

`src/pages/SettingsPage.tsx` - **Private account settings route /settings**

Requires a restored user/profile, with `SettingsForm` keyed by owner to prevent account-state mixing. Edits private names and optional sex, gender, kg/cm measurements and age, allows clearing values and stores a future-notification preference. Changes passwords through Better Auth with other-session revocation. Deletion uses a native confirmation dialog, current password and literal DELETE before server account deletion; client state is invalidated and navigation returns to search.

`src/styles.css` - **Global visual system**

One large stylesheet defines tokens, resets, typography, buttons, fields, layout, onboarding, care search, stylized map, clinic profile, charts, saved plans, report flow, About sections and responsive behavior. Later refinements and blue/orange account/admin rules extend earlier blocks. Media queries adapt widths; reduced-motion and RTL/font rules support access. CSS cascade order matters: a later token/selector can override an earlier one. Palette audit reads the final matching color tokens.

`src/types.ts` - **Shared product-domain contracts**

Defines visit modes, coordinates, stage estimates, historical records, confidence factors, reports, clinic hours/ratings/providers/reviews, search filters, saved plans, report drafts and toasts. Pages, service adapters and server modules share these interfaces. Types catch incompatible shapes during compilation; they are not runtime input validation, so Zod schemas remain necessary.

`src/vite-env.d.ts` - **Vite ambient type declaration**

References vite/client so TypeScript understands browser `import.meta.env` and Vite module conventions. It does not contain values or load an env file. Separate server type configuration keeps the Node runtime context explicit.

6. Browser state, services, shared logic and translations

`src/context/AppContext.tsx` - **Browser session and shared state coordinator**

`AppProvider` owns user/profile/admin readiness, saved IDs/plans, current submissions, language, tour and toasts. `refreshAccount` reads /me before protected UI appears; generations/request IDs/data revisions protect against stale identity and save snapshots. `clearPrivate` resets only MediQ state on account changes. Cross-tab broadcasts/storage signals, focus and 30-second checks restore sessions; visible interaction sends presence once per minute. Account saves/preferences use the API; guest personal changes stay in memory. `useApp` enforces provider use.

`src/context/expandedTranslations.ts` - **Named catalog assembler**

Imports six JSON locale files and exports the language-to-catalog map consumed by `translate`. Named language keys avoid the column-position mistakes possible in the older table. All imported catalogs are currently bundled eagerly, contributing to bundle size.

`src/context/interfaceTranslations.ts` - **Legacy translation compatibility table**

Large mapping from English source strings to Spanish/Chinese/Arabic arrays in a fixed column order. It remains a fallback for existing strings and is consumed by the audit. Positional storage is more error-prone than named catalogs; newer catalogs in `expandedTranslations` override it where supplied. Individual translation phrases are grouped as content data rather than reproduced.

`src/context/locales/ar.json` - **Arabic named translation catalog**

Contains 249 source-string-to-translation entries in this snapshot. Imported by `expandedTranslations.ts`, used by `translate` and checked by `i18n-audit`. Translates labels/messages/checklist copy and preserves named interpolation placeholders. Directory proper names, arbitrary notes and fixture review prose are not automatically translated. Individual phrases are content data grouped instead of copied into this guide; each catalog file is accounted for separately.

`src/context/locales/es.json` - **Spanish named translation catalog**

Contains 249 source-string-to-translation entries in this snapshot. Imported by `expandedTranslations.ts`, used by `translate` and checked by `i18n-audit`. Translates labels/messages/checklist copy and preserves named interpolation placeholders. Directory proper names, arbitrary notes and fixture review prose are not automatically translated. Individual phrases are content data grouped instead of copied into this guide; each catalog file is accounted for separately.

`src/context/locales/gu.json` - **Gujarati named translation catalog**

Contains 673 source-string-to-translation entries in this snapshot. Imported by `expandedTranslations.ts`, used by `translate` and checked by `i18n-audit`. Translates labels/messages/checklist copy and preserves named interpolation placeholders. Directory proper names, arbitrary notes and fixture review prose are not automatically translated. Individual phrases are content data grouped instead of copied into this guide; each catalog file is accounted for separately.

`src/context/locales/hi.json` - **Hindi named translation catalog**

Contains 673 source-string-to-translation entries in this snapshot. Imported by `expandedTranslations.ts`, used by `translate` and checked by `i18n-audit`. Translates labels/messages/checklist copy and preserves named interpolation placeholders. Directory proper names, arbitrary notes and fixture review prose are not automatically translated. Individual phrases are content data grouped instead of copied into this guide; each catalog file is accounted for separately.

`src/context/locales/pl.json` - **Polish named translation catalog**

Contains 673 source-string-to-translation entries in this snapshot. Imported by `expandedTranslations.ts`, used by `translate` and checked by `i18n-audit`. Translates labels/messages/checklist copy and preserves named interpolation placeholders. Directory proper names, arbitrary notes and fixture review prose are not automatically translated. Individual phrases are content data grouped instead of copied into this guide; each catalog file is accounted for separately.

`src/context/locales/zh.json` - **Chinese named translation catalog**

Contains 249 source-string-to-translation entries in this snapshot. Imported by `expandedTranslations.ts`, used by `translate` and checked by `i18n-audit`. Translates labels/messages/checklist copy and preserves named interpolation placeholders. Directory proper names, arbitrary notes and fixture review prose are not automatically translated. Individual phrases are content data grouped instead of copied into this guide; each catalog file is accounted for separately.

`src/context/translate.ts` - **Translation lookup and formatting**

Defines core symbolic navigation/hero keys, supported language codes and translate. Lookup checks named catalogs, base translations, then legacy positional entries, finally the source key. Interpolation replaces named placeholders and formats numbers with Intl. English generally uses source text; Polish/Gujarati/Hindi depend on named catalogs, not a legacy column. AppContext sets document.lang and Arabic rtl direction.

`src/data/mockData.ts` - **Single fictional product-data source**

Defines six clinic objects plus provider/review/history/report examples, insurance options, specialty options and language choices. history/report helpers convert compact fixture input to domain objects; ratings are derived from fictional reviews. clinicLanguageOptions derives distinct stable spoken-language values. sampleAppointment uses a future runtime time for a guest example. Used by mockClinicService, backend seed and selected explanatory pages; it is not authoritative healthcare data.

`src/services/account.ts` - **Private account response contracts**

Defines AccountUser, PrivateProfile and AccountState, including private sex/gender/measurements, saved IDs/plans and admin flag. These types describe /me and browser settings state; they do not grant roles or validate input. Optional fields and guest null user are deliberate.

`src/services/api.ts` - **Browser HTTP and session signal boundary**

api prefixes /api, uses same-origin cookies/no-store, a 15-second timeout, JSON bodies and an expected-account header for signed-in requests. ApiError keeps status/code and errorKey maps errors to display text. Unauthorized/account-change responses signal session invalidation. authRequest wraps auth POSTs. announceAccountChange uses an in-page event, BroadcastChannel and temporary storage signal. There are no browser database credentials or bearer session tokens managed here.

`src/services/clinicService.ts` - **Mode-selecting clinic data interface**

Declares search/detail/live/history/report/plan operations. mockClinicService applies deterministic search to fixtures, simulates delay and creates a temporary report shape. backendClinicService calls /clinics, /reports, /plans and /me, distinguishing detail 404 from outage. Per-draft UUID idempotency keys make retries stable; clearReportRequests resets them on identity change. Only the configured adapter is selected; backend errors never silently substitute fictional local results.

`src/services/locationService.ts` - **Permission-triggered approximate location**

requestApproximateLocation wraps browser geolocation only after Use my location, uses a timeout/low-accuracy option and rounds coordinates to two decimal places before returning them. Does not save coordinates or contact a map service. Backend searches receive that rounded origin to compute distance; this is still approximate geographic information in transit.

`src/services/search.ts` - **Shared deterministic directory search**

normalizeSearch handles case/accents/punctuation; relevance scores exact names, partial names, addresses, declared capabilities, curated multilingual concepts and bounded typo matches. editDistance supports restricted name fuzziness. searchDirectory enforces all hard filters before ranking; open/distance/time-of-day ties apply when relevant. distanceMiles uses Haversine geometry. Server and mock adapter use the same algorithm, avoiding divergent results. Unsupported concepts stay empty; no model or external AI request is involved.

`src/utils/navigation.tsx` - **Lightweight client-side router utilities**

`navigate` pushes/replaces history and dispatches `popstate`. `useLocationPath` listens to path/query/hash changes and handles scroll-to-top/anchor behavior. `Link` renders a genuine anchor while intercepting ordinary clicks for SPA navigation; `modifier` clicks retain browser behavior. `App.tsx` supplies route mapping; no React Router dependency is installed. The hosting server still needs route fallback for direct reloads.

`src/utils/report.ts` - **Shared draft and form validation helpers**

`localToday` handles browser timezone; `initialDraft` supplies empty fields, anonymous default and timezone offset. `reportRanges` defines permitted approximations. `validateReport` checks clinic/date, current elapsed bounds, exact ordered times or a known range and optionally full experience fields. Exact overnight times are rejected with a range suggestion. Server `validateDraft` adds trusted checks and duration derivation beyond these user-facing errors.

`src/utils/time.ts` - **Locale formatting and visit-plan arithmetic**

Formats dates, times and relative report times using the document language with Intl. `getVisitPlan` subtracts travel+buffer from visit start for leave-by and adds the selected mode estimate for likely finish, falling back to the first estimate. `toLocalDateTimeInput` converts a Date for a local form field without shifting displayed clock time. Manual travel input and fictional timing remain planning aids, not guaranteed arrival/completion.

7. Server files and database actions

`server/application.ts` - **Central API dispatcher and business actions**

`createApplication` composes database, Better Auth and seed checklist; handle routes Web Requests. Shared boundary enforces origin, JSON and persistent rate limits. Auth responses strip bearer tokens; private routes derive session ownership. Clinic reads combine stored JSON, accepted public reports and evidence. Report writes `validate/deduplicate/moderate/recalculate` in transactions. Admin reads/writes require verified role and enabled environment, and mutations append audits. Typed input/HTTP errors become explicit JSON codes; unexpected errors count service failures. See endpoint inventory and request traces.

`server/auth.ts` - **Better Auth and verification delivery**

`makeAuth` configures required verified email, no automatic sign-in after signup/verification, 12-128 character passwords, DB-backed 30-day sessions and secure `HttpOnly SameSite` cookies. OTP plugin uses six digits, ten-minute expiry, five attempts and hashed storage. Local transport writes private JSON; deployed transport calls Resend with timeout. HMAC email cooldown/daily limits guard delivery. Hooks create verified metadata/event counts and reject pending sessions; raw IP/user agent storage is disabled.

`server/cli.ts` - **Trusted operator command entry**

Reads server config and opens the database before dispatching `migrate`, `seed`, `admin`, `jobs` or `backup`. `Migrate` uses the shared `openDatabase` behavior; `seed` adds permitted fixtures and runs jobs. `Admin grant` requires an existing verified email and enabled environment. `Backup` uses SQLite online backup to include committed WAL data, then advises off-host encryption. Database always closes in finally. Even an operator command opens/migrates the chosen configured database.

`server/config.ts` - **Server environment validation**

`readConfig` validates environment, strong deployed secret, HTTPS origin, real email mode and required provider settings. Resolves database/mail paths and defaults, host/port, static serving, proxy trust and admin availability. It fails early rather than starting a seemingly working deployed app with unsafe/incomplete settings. Admin is disabled exactly in production; test/development/staging may support roles.

`server/database.ts` - **SQLite opening, migrations and writes**

`openDatabase` creates private directories, verifies an existing environment marker before migration, enables foreign keys/WAL/busy timeout and applies sorted SQL files once transactionally via migrations bookkeeping. transaction commits or rolls back blocks. `countEvent` upserts UTC daily metrics. `writeClinic` upserts rich clinic JSON and rebuilds related providers/languages/insurance plus source metadata. The browser never opens this database directly.

`server/estimator.ts` - **Conservative per-mode report estimator**

`estimate` separates accepted recent ongoing lower bounds from positive completed totals in seven days. Requires three completed samples, median/MAD outlier filtering and at least one fresh sample within 24 hours. Computes a recency-weighted mean/range and sample/freshness-based confidence. Returns insufficient/stale/ready explicitly and records excluded outliers. No ML model is trained; the algorithm is deterministic and testable.

`server/index.ts` - **HTTP process entry and optional static server**

Reads config, creates application/database/auth, optionally starts scheduled jobs, then accepts HTTP traffic. `/api` requests are buffered with a 32 KB limit and adapted to Web Request/Response, preserving multiple cookies. Other requests are served from dist only with `SERVE_STATIC`; path containment, content types, security/cache headers and extensionless SPA fallback apply. Forwarded IP handling depends on `TRUST_PROXY`. `SIGTERM/SIGINT` close the server/jobs/database. Development normally leaves static serving disabled.

`server/integrations.ts` - **Explicit server-only extension contracts**

Defines available/unavailable result types and optional directory, ratings, travel, booking and notification adapters. Registry is currently empty. `notifyWithConsent` requires both consent and a configured provider; otherwise returns unavailable. Typed hooks do not implement a vendor connection. Browser callers cannot register arbitrary URLs/keys; future approved adapters must be installed server-side.

`server/jobs.ts` - **Recalculation, retention and provider refresh**

`recalculate` computes each clinic/mode estimate and upserts current/history records. `runJobs` records runs and retries each job three times; retention removes expired auth/pending identities, presence, old anti-abuse links, old history/runs and local mail. An absent directory adapter records unconfigured, not successful refresh. `startJobSchedule` runs immediately and every five minutes without overlap, logging failures and allowing shutdown to await completion.

`server/migrations/001_initial.sql` - **Current 2.1.4 schema definition**

Creates Better Auth identity/session/account/verification tables and application profile, ownership, clinic, report, estimate, metrics, admin/audit/job tables with keys, constraints and deletion behavior. Applied once by `database.ts`; migrations table itself is created in `database.ts`. This branch contains only migration 001. The archived 2.1.5 migration 002 is not part of current source. See chapter 13 for every table and key relationship.

`server/security.ts` - **Common HTTP errors, HMAC identity and throttling**

`HttpError` carries status/code. `hash` uses SHA-256 HMAC so anti-abuse keys do not store plain email/IP. `limit` upserts a persistent fixed-window counter and throws 429 on excess. `anonymousIdentity` verifies a UUID/signature cookie with timing-safe comparison or generates a new signed one-day `HttpOnly` identity. Pseudonymous bucketing is not a guarantee of absolute anonymity or verified visits.

`server/seed.ts` - **Fictional directory and operational checklist seed**

`seedTasks` inserts twelve launch checklist items without resetting existing progress. `seedDemo` transactionally adds missing clinic fixtures with related metadata but refuses production and creates no accounts. It does not insert fixture `recentReports` as accepted evidence rows. Jobs and metrics distinguish fixture content from actual received submissions; rerunning seed does not overwrite existing clinic corrections.

`server/validation.ts` - **Runtime schemas and authoritative report checks**

Zod strict schemas accept only known `signup/profile/preference/plan/report/correction` fields, enforce lengths/ranges/enums and reject injected ownership fields. `validateDraft` confirms clinic/mode, real dates, bounded recent windows, timezone-aware future times and ongoing/completed durations, using shared form validation then stronger server checks. Corrections allow a defined directory subset rather than unrestricted replacement of clinic documents.

8. Root configuration, static assets, scripts, tests, deployment and existing documents

`.DS_Store` - **Tracked macOS Finder metadata**

A binary operating-system file with folder display information. It has no application, build, database or deployment purpose. Its binary contents are intentionally not decoded. It remains tracked despite its ignore rule; this documentation task does not remove it.

`.env.example` - **Environment template**

Tracked example configuration, not a loaded real credential file. Its exact keys, browser/server scope, required setup and differences by environment are documented in chapter 4. Do not deploy placeholder secrets; choose independent server credentials and matching exact browser origin. The `.example` suffix requires deliberate copying or equivalent injected values.

`.env.production.example` - **Environment template**

Tracked example configuration, not a loaded real credential file. Its exact keys, browser/server scope, required setup and differences by environment are documented in chapter 4. Do not deploy placeholder secrets; choose independent server credentials and matching exact browser origin. The `.example` suffix requires deliberate copying or equivalent injected values.

`.env.staging.example` - **Environment template**

Tracked example configuration, not a loaded real credential file. Its exact keys, browser/server scope, required setup and differences by environment are documented in chapter 4. Do not deploy placeholder secrets; choose independent server credentials and matching exact browser origin. The `.example` suffix requires deliberate copying or equivalent injected values.

`.gitignore` - **Untracked-file exclusion rules**

Ignores node_modules, dist, TypeScript build caches, Finder metadata, real env files, .data/database files, Playwright reports and README-LOCAL-SETUP.md. Negated rules retain environment examples. Ignore rules do not remove files already committed; this snapshot still tracks many dependency files, dist files, build caches and .DS_Store.

README.md - **Existing quick project guide**

The preserved entry-level overview documents local startup, commands, product routes, deliberate persistence changes, important paths and deployment boundaries. This new onboarding README expands it without replacing it. The new documents are self-contained and can be read before following the shorter guide.

deploy/render.production.yaml - **Prepared public release hosting template**

Describes an independent production web service/disk with build:production and server production mode, which disables admin. Uses distinct credentials/configuration and explicit owner-provided origin/email settings, with automatic deploy off. Hosting YAML is a template, not a provisioned global address. Current fictional seed policy means production does not automatically display the six demo clinics.

deploy/render.staging.yaml - **Prepared internal release hosting template**

Describes a separate Node staging web service, paid persistent disk mounted at /var/data, build:staging, npm run api, /api/health, Node 24, jobs, proxy/static settings and private required values. Auto deploy trigger is off. AUTH_SECRET is generated by host; origin/email values require owner setup. Its name does not provide access protection to the whole staging site.

docs/backend-backlog.md - **Implemented status and external launch backlog**

Summarizes the 2.1.4 backend already implemented and the remaining hosting/provider/data/backup/review/private-operations work. Separates real local functionality from unconfigured vendor integrations. Useful next to runbook, not a replacement for reading current handlers.

docs/data-and-estimates.md - **Data ownership and algorithm definitions**

Defines deletion, guest/account persistence, report privacy, rate limits, estimation thresholds, search semantics, metric windows and localization boundaries. Most operational details align with current code. Describes backup/vendor responsibilities and the difference between pseudonymous anti-abuse keys and true anonymity. Read with database, validation, estimator and jobs.

docs/next-version-implementation-prompt.md - **Historical implementation request**

Original detailed scope/acceptance prompt for full-stack upgrade, language/palette, accounts, environments, admin, search and operational tasks. It is design input, not executable configuration or evidence every future/provider requirement is complete. Existing preservation rules explain why service adapters and route compatibility matter.

docs/product-background.md - **Retained earlier product/research context**

Stores rationale, hypotheses and pilot questions moved out of the patient-facing About page. Some descriptions are historical: claims of browser-local persistence or inactive anti-abuse controls do not describe the current backend. Account synchronization is implemented now. Use it for product background, then verify behavior in current code and data-and-estimates.md.

`docs/runbook.md` - **Operator setup/release/recovery reference**

Documents frontend/backend architecture, one-time setup, local mailbox, trusted admin bootstrap, staging/production separation, hosting/email requirements, job scheduling, backup/rollback and verification commands. It is also read by `/api/admin/guide` for authorized users, creating a real runtime dependency from API code to this documentation file. Cloud steps are instructions, not proof of provisioned services.

`docs/upgrade-checklist.md` - **Historical requirement coverage record**

Maps earlier upgrade requirements to implementation/tests and distinguishes local completion from external owner work. Contains historical branch/time context, not the current main branch name. Useful for finding intended guarantees and remaining launch responsibilities; code and new snapshot inventory remain current authority.

`docs/verification.md` - **Historical test evidence and limits**

Records earlier successful checks, fixes, warnings and external tasks. Notes generated tracked dist/caches were restored after verification, so a fresh build is needed for updated preview output. Reports past test execution and tool versions; it is not a continuously updated test dashboard or current vulnerability scan.

`index.html` - **Browser entry document**

Provides metadata, favicon, viewport, theme color, a keyboard skip link, the root mount element and module script for `src/main.tsx`. Vite rewrites the script reference in built output. Deep-link hosts must serve this document for application routes because route selection happens in React.

`package-lock.json` - **Reproducible dependency graph**

Locks resolved package versions, package locations and integrity information for npm ci. Caret ranges in `package.json` express allowed versions; this lockfile pins the installed graph for reproducible builds. Thousands of lock entries are generated bookkeeping, grouped rather than explained line by line. Changing it affects future installs even if no UI source changes.

`package.json` - **Dependency and command manifest**

Declares an ES-module, private application package; runtime dependencies, development tools, Node engine floor and all npm scripts. Its package version is 0.1.0, which is distinct from the Git release name 2.1.4. This file defines commands; it does not run them until npm is invoked. See chapter 12 for every script and chapter 9 for each dependency.

`playwright.config.ts` - **Browser automation configuration**

Runs tests/browser with one worker, controlled timeouts and base URL `localhost:4173`. Starts a dedicated test API through `e2e-server.ts` and Vite with backend/staging browser flags; existing servers are not reused. Produces list and HTML reports; failure screenshots/traces are conditional. The API's test environment and browser's staging flag serve different purposes.

`public/_redirects` - **Static SPA redirect rule**

Contains a catch-all rewrite to `index.html` for hosts that understand this format. Vite copies it to dist. Like `vercel.json` it addresses browser deep links, not an API/database. `server/index.ts` has its own SPA fallback when static serving is enabled.

`public/favicon.svg` - **Browser tab icon**

Static vector brand icon referenced by `index.html` and copied without compilation to `dist/favicon.svg`. It is separate from the inline SVG/wordmark in `Brand.tsx`. Editing one does not automatically update the other.

`scripts/build-release.mjs` - **Environment-specific release build and isolation check**

Validates staging/production selection, runs TypeScript and Vite with forced backend/environment flags, then inspects dist JS. Staging must contain an `AdminPage` chunk; production must not contain that chunk or internal account/checklist endpoint strings. Assertions fail the build if isolation is wrong. This creates a release artifact, not an actual deployment.

`scripts/e2e-server.ts` - **Isolated browser-test backend launcher**

Overrides environment/database/mail/origin/port for test, removes only `.data/e2e`, initializes schema/fixtures/jobs, closes that initial DB and starts `server/index.ts`. Recreating test data makes browser scenarios repeatable and independent of `.data/development`. It is destructive only to its dedicated generated `e2e` directory, not a normal startup script.

`scripts/il8n-audit.mjs` - **Translation and interpolation coverage check**

Uses the TypeScript parser to collect literal and conditional `t(...)` keys in TSX and strings from seeded checklist tasks; checks all six catalogs plus legacy compatible columns and core symbolic keys. Ensures placeholder sets match source strings. It does not prove translation quality or cover arbitrary future dynamic strings; native-language review is separate.

`scripts/palette-audit.mjs` - **Core color contrast check**

Reads final CSS token values, computes WCAG relative luminance and checks selected text/background pairs against 4.5:1. Reports each ratio and fails if one is below the threshold. This is a narrow automated color check, not a complete accessibility certification.

`scripts/serve-dist.mjs` - **Local static build preview helper**

Creates a loopback HTTP server for dist with basic MIME/cache handling and SPA fallback. Port defaults 4173 or `PORT`. Used by smoke tests and `serve:dist`; it does not dispatch `/api` to `application.ts` or proxy the backend. A successful static preview alone cannot prove account/report API behavior.

`scripts/smoke-test.mjs` - **Built routes and shared product regression checks**

Starts `serve-dist` on 43190, checks SPA route fallback and built CSS/JS assets, verifies fixture IDs and uses a Vite SSR module loader to exercise mock service/search/time/report logic. Selects `mockClinicService` explicitly to avoid backend environment confusion. Assertions cover filters, location geometry, stage totals, reports, planning and translation table shape. Cleans up helper server in finally.

`tests/backend.test.ts` - **Real API/auth/SQLite integration tests**

Node test runner creates temporary isolated SQL databases and cookie-aware in-process clients that call `app.handle` with realistic headers. Six groups cover auth lifecycle/isolation/deletion/persistence, OTP/recovery/rate limits, reports/admin/checklist, estimator, search and origin/production/jobs/backup boundaries. Test mail messages are read only in temporary fixtures to retrieve OTPs; no real user mailbox is required.

tests/browser/mediq.spec.ts - **End-to-end browser interaction tests**

Seven Playwright scenarios operate real Vite/API test servers and separate browser contexts/tabs. Cover guest care navigation/search/map/reports/temporary saves; verified accounts and second-device state; seven languages/RTL; stale search/retry; admin authorization; routes/responsive/dialog keyboard; and delayed account refresh/logout/switching races. Test helper uses only the e2e database/mail. Screenshots and reports are generated diagnostic artifacts.

tsconfig.app.json - **Browser TypeScript compilation scope**

Checks src with ES2022+DOM libraries, strict mode, React automatic JSX, JSON imports, bundler resolution and vite/client types. noEmit makes this a type check; Vite handles actual browser compilation. Explicit ambient types avoid accidentally including unrelated installed type packages.

tsconfig.app.tsbuildinfo - **Tracked incremental compiler cache**

Generated metadata used to accelerate repeated browser type checks. Not human-authored logic or a deployable script. Git still tracks it in this snapshot despite *.tsbuildinfo ignore rules. Contents are grouped and not decoded entry by entry.

tsconfig.json - **TypeScript project references**

Root build configuration has no direct source files and references browser and Vite-config projects. tsc -b follows this graph. Server/tests are checked through the additional explicit server config in the typecheck command, not this root reference list.

tsconfig.node.json - **Vite-config TypeScript scope**

Checks vite.config.ts with Node types, bundler resolution and no emitted JavaScript. Composite mode participates in the root reference build. Node-specific config is distinct from browser DOM files.

tsconfig.node.tsbuildinfo - **Tracked config compiler cache**

Generated metadata for the Node/Vite configuration project. Can change after a typecheck even when source is unchanged. Like the app cache it is historically tracked, regenerable and not authoritative configuration.

tsconfig.server.json - **API and test TypeScript scope**

Checks server/**/*.ts and tests/**/*.ts with ES2023+DOM declarations, Node types, strict mode and explicit .ts import support. DOM here supplies Web Request/Response type declarations used by modern Node; it does not turn the server into a browser process. noEmit means tsx runs TypeScript source at runtime.

vercel.json - **Static route fallback template**

Rewrites paths to index.html so a static deployment can reload React routes. It is a hosting hint, not deployment evidence or backend provisioning. It does not configure a permanent SQLite volume, email delivery or the Node API, and is not a complete hosting solution for backend mode.

vite.config.ts - **Development server and browser bundler configuration**

Loads VITE_ environment settings, enables the React plugin, sets dev and preview port 4173 and proxies development /api requests to the loopback API on 3001. It forces production/staging build environment flags based on Vite mode to prevent accidental public admin inclusion. The development proxy is not an API server and is not automatically reproduced by a static host.

Server environment template cross-reference

The three server env examples are cataloged here as well as in chapter 4 so no tracked file is omitted.

server/.env.example - Environment template

Tracked example configuration, not a loaded real credential file. Its exact keys, browser/server scope, required setup and differences by environment are documented in chapter 4. Do not deploy placeholder secrets; choose independent server credentials and matching exact browser origin. The .example suffix requires deliberate copying or equivalent injected values.

server/.env.production.example - Environment template

Tracked example configuration, not a loaded real credential file. Its exact keys, browser/server scope, required setup and differences by environment are documented in chapter 4. Do not deploy placeholder secrets; choose independent server credentials and matching exact browser origin. The .example suffix requires deliberate copying or equivalent injected values.

server/.env.staging.example - Environment template

Tracked example configuration, not a loaded real credential file. Its exact keys, browser/server scope, required setup and differences by environment are documented in chapter 4. Do not deploy placeholder secrets; choose independent server credentials and matching exact browser origin. The .example suffix requires deliberate copying or equivalent injected values.

9. Dependencies, installation and grouped external files

package.json expresses direct dependencies; package-lock.json records the resolved graph; node_modules is installed executable/library content. They have different jobs. Packages listed as development dependencies may still be needed on the deployed service because npm run api directly uses tsx. An install that omits all development dependencies would break that current start command unless the runtime/build strategy is changed.

The direct dependencies at this source snapshot are:

Package	Declared range/version	Role
better-auth	^1.7.6 (dependencies)	Server passwords, verification OTP plugin, sessions and secure cookie handling; configured in server/auth.ts.
lucide-react	^1.31.0 (dependencies)	React SVG icon components imported throughout the interface; not a map or data provider.
react	19.2.6 (dependencies)	Component rendering, hooks, effects and shared context.
react-dom	19.2.6 (dependencies)	DOM mounting and portals, including native modal rendering.
zod	^4.6.5 (dependencies)	Runtime validation of API inputs and configuration-shaped data.
@playwright/test	^1.63.0 (devDependencies)	Browser scenario runner, assertions, managed test servers and diagnostics.

Package	Declared range/version	Role
@types/node	^26.6.3 (devDependencies)	Type declarations for Node, including its runtime APIs.
@types/react	19.2.14 (devDependencies)	React TypeScript types.
@types/react-dom	19.2.3 (devDependencies)	React DOM TypeScript types.
@vitejs/plugin-react	6.0.2 (devDependencies)	React transform support in Vite development/builds.
tsx	^4.23.15 (devDependencies)	Runs the API and CLI TypeScript sources directly under Node.
typescript	5.9.3 (devDependencies)	Type checks/build metadata and AST parsing for translation audit.
vite	^8.2.2 (devDependencies)	Development server, /api proxy, browser production bundler and smoke-test module loader.

SQLite is supplied by Node's `node:sqlite`, not a separate npm SQLite driver in this manifest. The code uses synchronous DB operations inside a single API process. Better Auth itself pulls other adapters, query/crypto/HTTP utilities and schema packages; having an adapter installed does not mean MediQ uses its database vendor.

9.1 Every installed top-level package family

The following grouped inventory covers all installed package roots present locally, including scoped children and duplicate folders with a space/number suffix. These groups replace line-by-line explanations of external source, compiled distributions, type declarations and licenses. Dependencies change when the lockfile/install changes.

`node_modules/@better-auth/`

Authentication core, utility, telemetry and optional database adapters; current app supplies SQLite to Better Auth. Prisma, Drizzle, Mongo and memory adapter packages being present is not evidence of use.

Installed children: core, core 2, drizzle-adapter, drizzle-adapter 2, kysely-adapter, memory-adapter, mongo-adapter, prisma-adapter, telemetry, utils, utils 2.

`node_modules/@better-fetch/`

HTTP request utility used in the authentication dependency stack.

Installed children: fetch.

`node_modules/@esbuild/`

Platform-specific esbuild executable support for transforms.

Installed children: darwin-arm64.

`node_modules/@noble/`

Cryptographic hashes/ciphers used by authentication/security dependencies.

Installed children: ciphers, hashes.

`node_modules/@opentelemetry/`

Telemetry vocabulary used by dependencies; installed code is not proof of a configured monitoring exporter.

Installed children: semantic-conventions, semantic-conventions 2.

node_modules/@oxc-project/

Compiler/tooling type contracts used in the build stack.

Installed children: types.

node_modules/@playwright/

Playwright test wrapper; browser engine automation lives in playwright/playwright-core.

Installed children: test.

node_modules/@rolldown/

Bundler native binding and plugin utilities for Vite.

Installed children: binding-darwin-arm64, pluginutils, pluginutils 2.

node_modules/@standard-schema/

Shared schema interface contracts used by validation dependencies.

Installed children: spec.

node_modules/@types/

Type declarations only, not executing runtime packages.

Installed children: node, node 2, react, react-dom.

node_modules/@vitejs/

Vite React plugin.

Installed children: plugin-react.

node_modules/better-auth/

Configured authentication provider library.

node_modules/better-call/

Routing/request handler utility underlying authentication.

node_modules/csstype/

CSS property types for React tooling.

node_modules/defu/

Dependency configuration-default merging.

node_modules/detect-libc/

Native-package platform/libc detection.

node_modules/esbuild/

Fast JavaScript/TypeScript transform tool used in tooling.

`node_modules/fdir/`

Filesystem crawler used by build globbing.

`node_modules/fsevents/`

macOS filesystem-event support for watching.

`node_modules/jose/`

JWT/JWS/JWE cryptographic utilities in the auth dependency graph.

`node_modules/kysely/`

SQL query builder used by authentication data adapters.

`node_modules/lightningcss/`

CSS processing and optimization tooling.

`node_modules/lightningcss-darwin-arm64/`

Native CSS engine for the local Apple Silicon platform.

`node_modules/lucide-react/`

Bundled icon component library.

`node_modules/nanoid/`

Small unique-ID generator used by tooling/dependencies.

`node_modules/nanostores/`

Small state/store utility used by dependencies; app shared state is React context.

`node_modules/picocolors/`

Terminal color formatting.

`node_modules/picomatch/`

Glob pattern matching.

`node_modules/playwright/`

Browser automation API and installation tools.

`node_modules/playwright-core/`

Browser driver/automation internals.

`node_modules/postcss/`

CSS parse/transform infrastructure.

`node_modules/react/`

React runtime.

`node_modules/react-dom/`

DOM renderer.

`node_modules/rolldown/`

JavaScript bundler used by Vite.

`node_modules/rou3/`

Router utility in auth/HTTP dependencies.

`node_modules/scheduler/`

React task scheduling.

`node_modules/set-cookie-parser/`

HTTP cookie header parsing for dependencies.

`node_modules/source-map-js/`

Source-map processing for build tooling.

`node_modules/tinyglobby/`

Fast file glob expansion for tooling.

`node_modules/tsx/`

TypeScript runtime loader.

`node_modules/typescript/`

Compiler/parser/type service.

`node_modules/undici-types/`

Node HTTP/fetch type declarations.

`node_modules/vite/`

Development and build framework.

`node_modules/zod/`

Runtime schema validation.

9.2 Hidden dependency folders and content types

- `node_modules/.bin/`: executable shims/symlinks for npm scripts. Present tools: esbuild, nanoid, playwright, playwright-core, rolldown, tsc, tsserver, tsx and vite. These link into packages; they are not separate authored application programs.
- `node_modules/.package-lock.json`: npm's local installed-tree bookkeeping, separate from root lockfile.
- `node_modules/.vite/`: Vite optimized dependency cache; `deps/` contains generated JS, maps, metadata and its own package.json. Local `deps 2/` is duplicate cached content, not a second application. Changes here can occur simply by starting dev.

- node_modules/.vite-temp/: temporary transformed Vite configuration/cache area; empty at inspection.
- Package dist/, lib/, src/, types/, nested dependencies and platform binaries: external implementations distributed by package authors. Their nested folder purposes inherit this group explanation.
- Package README/license/package.json files: external usage/legal/manifests, not MediQ documentation or product data. Type files describe library contracts. Source maps map generated code to package source. Native .node/executable files match operating system and CPU architecture.
- Folders with names such as node 2, core 2, utils 2, pluginutils 2 and semantic-conventions 2 are present local duplicates. They are not imported intentionally by MediQ source. Explicit compiler types reduce unintended ambient-type discovery. Their origin is not established from their names alone.

This snapshot tracks 4,643 node_modules paths in Git, although .gitignore excludes new dependency paths. It also has additional ignored installed packages. This is historical repository content, not a recommendation to version installed libraries. No dependency cleanup, upgrade or license evaluation is performed by this documentation task.

10. Hidden folders, local files and generated output

10.1 .git/: repository mechanics, never website assets

Git stores revision history separately from working source. A branch is a reference to a commit; it is not a second directory of code. main and release branches can share commits. A revert adds an inverse change to history; it does not erase the original commit. This explains why 2.1.5 appears in history while current main files match 2.1.4.

Path or family	Purpose
.git/HEAD	Identifies the currently checked-out branch or detached commit.
.git/config	Local remote/tracking/repository settings. Not application environment or public hosting config. Its contents are not copied into this report.
.git/index	Active staging index used by git add/commit; binary bookkeeping.
.git/index 2, .git/index 3	Additional binary files present locally; Git's active index is the unsuffixed index. Exact origin of duplicates is not established.
.git/packed-refs	Compact storage for branch/tag references. A reference need not have a separate loose file.
.git/FETCH_HEAD	Most recently fetched remote references; updated by fetch.
.git/ORIG_HEAD	Previous head recorded by operations such as merge, helpful for recovery.
.git/AUTO_MERGE	Git-maintained auto-merge reference/state from merge tooling. Its presence alone is not proof of an active unresolved merge.
.git/COMMIT_EDITMSG	Last prepared commit-message text.
.git/description	Repository description, often a default Git value; not MediQ marketing copy.
.git/info/exclude	Checkout-specific ignore additions, unlike tracked .gitignore.

Path or family	Purpose
<code>.git/objects/00</code> through <code>.git/objects/ff</code>	Hash-prefix buckets for compressed loose commits, trees and file blobs; each has the same function.
<code>.git/objects/pack/</code>	Packed history storage: <code>.pack</code> data, <code>.idx</code> lookup and <code>.rev</code> reverse index. Current pack is <code>pack-244c2169bf59bc9da3752feb14f823358a5a2054</code> with those three extensions.
<code>.git/objects/info/</code>	Auxiliary object database metadata; empty at inspection.
<code>.git/refs/heads/</code>	Loose local branch pointers: <code>main</code> , <code>v2.1.4</code> , <code>v2.1.5</code> , <code>2.1.5</code> ; <code>codex/</code> holds the two older local upgrade branch refs.
<code>.git/refs/remotes/origin/</code>	Cached remote-tracking pointers and HEAD, not live automatic sync with GitHub.
<code>.git/refs/tags/</code>	Loose tag pointers; no files present in this folder at inspection.
<code>.git/logs/HEAD</code>	Local checkout/HEAD reflog history.
<code>.git/logs/refs/heads/</code> , nested <code>codex/</code>	Local branch reflogs corresponding to branch pointers.
<code>.git/logs/refs/remotes/origin/</code>	Remote-tracking reflogs; recover recent local reference movements.
<code>.git/hooks/</code>	Sample scripts for Git events. All present hooks have <code>.sample</code> suffix and are inactive by that name.

Every sample hook is accounted for: `applypatch-msg`, `commit-msg`, `fsmonitor-watchman`, `post-update`, `pre-applypatch`, `pre-commit`, `pre-merge-commit`, `pre-push`, `pre-rebase`, `pre-receive`, `prepare-commit-msg`, `push-to-checkout`, `sendemail-validate` and `update`. Samples demonstrate validation, formatting, notifications or integration at Git events; none implements website runtime behavior. Nested refs, logs/refs, heads, remotes, origin and codex are organizing namespaces for pointers/history, not deployment services. `.git` contents are grouped because compressed history/index internals are not authored application code.

10.2 `.data/`: local operational state

This folder is ignored and must never be a static web root. Databases and mail contain private data. The report documents file roles and schema, not user records or message contents.

- `.data/development/mediq.sqlite`: restored 2.1.4 development database used by current server settings. Current schema bookkeeping contains migration 001; the profiles schema includes gender, as 2.1.4 expects.
- `.data/development/mediq.sqlite-wal`: SQLite write-ahead log, potentially containing committed changes not checkpointed into the main file.
- `.data/development/mediq.sqlite-shm`: SQLite shared-memory coordination/index file for WAL readers/writers.
- `.data/development/verification-backup.sqlite` and its `-wal/-shm`: older local consistent-backup database and runtime sidecars; not the active configured database.
- `.data/development/mail/`: private local email transport. Empty at inspection; future UUID-named JSON messages may include recipient, subject, text, OTP, purpose and timestamp. This is why local signup does not send inbox email.
- `.data/e2e/`: generated browser-test-only database, WAL/SHM sidecars and `mail/`. Four local test messages were present at inspection. `e2e-server` deliberately recreates this folder for tests.
- `.data/archives/`: rollback preservation, outside the live app data path.

- `.data/archives/2026-10-08-v2.1.5/`: version transition backups. `current-development.sqlite` is an online-consistent backup of later data; `original-development.sqlite` and its WAL/SHM preserve the original moved files; `pre-v2.1.5-backup.sqlite` and its sidecars preserve the compatible earlier snapshot; `restore-record.json` records refs and restoration choice without private records.

Only filename/type/schema metadata is included. Individual SQLite pages, account/report records, private emails and OTPs are excluded explicitly. Do not copy only a live `.sqlite` file to make a backup: use SQLite's online backup command so WAL content is included. Code rollback and schema rollback must be coordinated; the 2.1.5 schema removed gender, so switching source alone previously left 2.1.4 incompatible. The archive remains preserved and is not the active database.

10.3 `dist/`: compiled browser distribution

`dist/index.html` is built HTML that references hashed bundles, `dist/favicon.svg` is copied branding and `dist/_redirects` is a copied SPA host hint. `dist/assets/` contains bundled JavaScript and CSS. Current tracked paths are listed in appendix A; additional local bundles from verification coexist. Hashed names change with content.

Five `dist` files are tracked historically. A clean Git status does not guarantee those checked-in bundles were rebuilt for current source: verification restored tracked generated artifacts previously. Source files and a fresh build are the authority. Do not debug or edit minified bundles as application source. `public/` is build input; `dist/` is build output.

10.4 Test results and other root local files

`playwright-report/index.html` is a generated HTML results viewer. `test-results/.last-run.json` records the test runner outcome. `test-results/signup-320.png`, `signup-390.png`, `signup-768.png` and `signup-1440.png` are captured responsive-signup screenshots. Failure trace ZIPs or screenshot subfolders may be generated in future runs; they can contain test interaction details and are not shipped as website assets.

`README-LOCAL-SETUP.md` is an ignored personal local/cloud setup guide. Its headings include startup, safe checkout upgrade, staging/email setup and deliberate later production. It can reflect the later upgrade period and should not override current 2.1.4 source/schema; do not run an absent migration just because older instructions mention it. Contents were not copied wholesale because a local setup file is not guaranteed free of private settings.

`.DS_Store`, compiler caches and `root/installed` lockfiles are explained individually in the tracked catalog. No additional first-party hidden project folders were present. Global developer tool folders outside the repository are not part of the requested codebase and are not inventoried.

11. End-to-end request and state walkthroughs

11.1 Initial load and navigation

1. Vite (development) or the static-serving API process (hosted mode) returns `index.html` and browser JS/CSS.
2. `main.tsx` mounts App. `AppProvider` initializes guest memory and, in backend mode, requests `GET /api/me`.
3. `api.ts` sends same-origin cookies. Locally Vite proxies `/api` to `server/index.ts`; the browser does not need to call port 3001 directly.
4. `application.ts` obtains a verified session from Better Auth. A guest gets `null user/admin false`; an account gets `private profile/saved IDs/plans`.
5. `AppRoutes` shows a loader or retry while account state is unresolved, then maps the current path to its screen. Layout renders navigation and toasts for non-landing routes.

6. Link/navigate change browser history without a full page reload. Back/forward and popstate update `useLocationPath`. Direct reloads still require the host's `index.html` fallback.

11.2 Signup, verification, login and logout

Signup page -> `authRequest` -> `POST /api/auth/sign-up/email` -> `origin/JSON/rate` checks -> `signupSchema` -> Better Auth -> pending user and hashed-password account -> OTP send callback. Development writes a private message file; Resend mode makes a provider request. The stored verification value is hashed; the local mailbox contains the actual code, so it is sensitive.

Verification page -> `POST /api/auth/email-otp/verify-email` -> provider expiry/attempt/one-use checks -> `emailVerified` update -> `account_metadata` insertion and `verified_signups` count. Verification does not automatically sign in. Pending identities cannot get private sessions.

Login -> `POST /api/auth/sign-in/email` -> verified credentials -> server-backed session cookie -> client `announceAccountChange` -> `refreshAccount` -> `/api/me` -> private state restored. Cookie is `HttpOnly`, so app code does not store a readable bearer token. API wrapper strips auth response token fields while preserving `Set-Cookie`.

Logout clears current private UI immediately, posts sign-out and broadcasts identity change. Other tabs clear/revalidate their state. Focus and 30-second visible-page refresh cover missed signals. Generation checks prevent a delayed old-owner response from putting private data back on screen after logout or account switching.

Password reset requests a separate OTP then resets password and revokes sessions. Password change checks current password and revokes other sessions. Account deletion requires the password and `DELETE` confirmation; a transaction deletes identity/owned rows, verification records in scope and recalculates estimates. Aggregate event counts remain. These flows reuse Better Auth rather than inventing a second password/session implementation.

11.3 Clinic search and optional location

`FindCarePage` updates `SearchFilters` -> `clinicService.searchClinics` -> backend adapter encodes filters as JSON in the `/clinics` query -> server parses strict filter schema -> reads stored clinic JSON and combines evidence/feed -> `searchDirectory` applies constraints and relevance -> JSON array -> request guard accepts only latest response -> memoized explicit sort -> `cards/map`.

In browser-only mode, `mockClinicService` calls the same `searchDirectory` with fixture objects after a simulated delay. No search request reaches a server. Backend mode failure has an error/retry state; it does not swap to mock data.

Use my location triggers geolocation permission -> rounds latitude/longitude -> stores approximate origin in filter memory -> search computes Haversine straight-line miles. The API does not save search/location history. The rendered map still uses fixture x/y positions, not the visitor's actual road map. Insurance, specialty and spoken-language labels use stable stored values independent of interface translation.

11.4 Saving a clinic or visit plan

A saved-clinic click -> `AppContext.toggleSavedClinic` -> owner present? If yes, `PUT/DELETE /api/saved-clinics/:id` -> `requireUser` -> clinic exists -> insert/delete under authenticated user ID. If guest, only memory changes. The context updates after successful account write and displays a toast, with per-item locks and generation guards.

`ClinicPage` plan dialog constructs `SavedAppointment` with unique ID, clinic, ISO visit time, mode and manual travel/buffer. Account save goes through `clinicService` -> `PUT /api/plans` -> strict `planSchema` -> supported clinic/mode -> owner-scoped SQL upsert and first-creation metric. Guest saves remain memory. `SavedPage` loads related clinics and uses `getVisitPlan`:

```
leave-by = visit-start - travel-minutes - buffer-minutes
```

```
arrive-by = visit-start - buffer-minutes
likely-finish = visit-start + selected total-visit estimate
```

Plans are not clinic reservations. They do not write to a provider calendar. Reloading a guest resets personal state to defaults, including the sample plan; an account restores persisted plans through /me.

11.5 Full/quick report, review and estimate update

ReportPage or QuickReport -> shared draft/form validation -> AppContext.submitReport -> clinicService -> stable Idempotency-Key -> POST /api/reports. Server gets account/anonymous actor, checks existing key, consumes actor/network rate limits, validates clinic/mode/dates/duration and checks a content fingerprint for recent duplicates.

Backend public report deliberately omits free-text notes, exact input times, names, emails and private profile fields. It always has anonymous=true publicly even if the UI anonymity control is unchecked. Reports above eight hours are marked review; other valid reports are accepted. SQL insert, counters and estimate recalculation happen in one transaction. The anonymous signed cookie identifies a repeat guest pseudonymously; it is not proof that the visit occurred.

Accepted results are merged into context for immediate display and public clinic reads retrieve accepted SQL reports. Review results are not shown in public feeds or estimate evidence. Admin acceptance/rejection is a one-way decision from review with reason/audit and recalculation. Duplicate retries return the previous result rather than create another report/event.

Ongoing waits store elapsed minutes and SQL total_minutes=NULL. Their JSON compatibility shape has totalMinutes=0, but the estimator never treats that as a completed zero-minute visit. Completed reports store a positive total; exact form times or a range midpoint yield that total.

11.6 Estimation and background work

For each clinic and supported visit mode, estimator reads accepted evidence. It uses completed totals from seven days, at least three samples, median/MAD outlier removal and at least one eligible report less than 24 hours old. Recency weights decay on a 48-hour scale. Weighted mean is rounded; spread includes weighted variability and a small-sample margin. Confidence rises with eligible/fresh sample counts and relative spread.

Under three eligible samples -> insufficient; old-only evidence -> stale; enough recent consistent evidence -> ready. Recent ongoing reports within two hours are summarized separately as count/longest wait, not mixed into completed averages. jobs.recalculate upserts estimates plus hourly estimate_history. HistoricalChart still uses fixture historicalWaits, not this SQL history.

Report insert, moderation and account deletion can recalculate immediately. RUN_JOBS=true enables an immediate run and a five-minute non-overlapping loop for freshness, retention and possible directory refresh. Otherwise npm run jobs is a deliberate operator command. Expiry is enforced by auth rules, but physical row/file deletion depends on jobs running.

11.7 Settings, internal administration and presence

Settings updates profile through PUT /profile and language through /preferences, scoped to session owner. Optional personal fields remain private and are not included in routine admin directory or report feed. Notification preference is a stored opt-in for future integration, not proof messages are sent.

Trusted server CLI admin:grant looks up a verified account then inserts its role in administrators. Signup/settings cannot grant that role. AdminPage calls protected endpoints for directory/metrics/tasks/review/corrections/audit. Account directory searches verified users only, 20 per page. Presence heartbeats occur for visible sessions with recent interaction; active-window metrics use five minutes and recent event totals use seven UTC dates. Sessions are not unique humans; confirmedBookings is null.

Admin correction, task edits and report decisions append audit records transactionally. Production excludes the page import and server role access independently. A separately hosted staging frontend is still public unless actual site access protection is implemented; role-gating admin screens does not protect every staging route.

12. Every npm command, build path and test boundary

12.1 Command reference

These commands are defined in the existing manifest. "Setup" is not a command to rerun on every page visit.

npm script	Underlying command	When and what it does
dev	vite	Normal local session: starts Vite source server on 4173. Use -- --force to refresh optimized dependency cache after switching versions.
build	tsc -b && vite build	Build/type check: tsc root projects then Vite production-mode browser bundle into dist. Does not separately run server typecheck or deploy.
preview	vite preview	Local Vite static preview of dist. Current config has no preview /api proxy; do not assume backend account calls work through this preview alone.
serve:dist	node scripts/serve-dist.mjs	Local custom static-only dist helper; not the backend.
typecheck	tsc -b && tsc -p tsconfig.server.json	Checks browser/Vite root projects, then server and test compiler scope without emitting server JS.
test	npm run test:il8n && npm run test:palette && npm run build && npm run test:smoke	Runs translation audit, palette audit, build and smoke sequentially. Backend and browser tests are separate commands.
test:smoke	node scripts/smoke-test.mjs	Checks built route fallback/assets and shared product logic using mock service.
api	node --import tsx --env-file-if-exists=server/.env server/index.ts	Normal backend session: starts Node API with server/.env if present; opens DB and applies pending numbered migrations.
db:migrate	node --import tsx --env-file-if-exists=server/.env server/cli.ts migrate	One-time setup or new migration: open chosen DB and apply absent migrations. Also happens on API startup.
db:seed	node --import tsx --env-file-if-exists=server/.env server/cli.ts seed	One-time permitted fictional demo setup: add missing fixture clinics/checklist, run jobs. Refuses production; creates no user accounts.
admin:grant	node --import tsx --env-file-if-exists=server/.env server/cli.ts admin	Trusted local/staging operator action: append --verified-email to grant a role. Production denies this.
jobs	node --import tsx --env-file-if-exists=server/.env server/cli.ts jobs	Manual recalculation/retention/adaptor job run on configured database.

npm script	Underlying command	When and what it does
db:backup	<code>node --import tsx --env-file-if-exists=server/.env server/cli.ts backup</code>	Pre-change protection: append --/private/new-backup.sqlite to create a consistent SQL backup. Use a new private destination outside static files.
test:backend	<code>node --import tsx --test tests/backend.test.ts</code>	Six real API/auth/database test groups in isolated temporary databases.
test:browser	<code>playwright test</code>	Seven Chromium browser scenarios; dedicated test servers and recreated .data/e2e.
build:staging	<code>node scripts/build-release.mjs staging</code>	Forced backend/staging release plus admin-inclusion assertions.
build:production	<code>node scripts/build-release.mjs production</code>	Forced backend/production release plus admin-exclusion assertions.
test:i18n	<code>node scripts/i18n-audit.mjs</code>	Checks known literal/conditional translations and interpolation placeholders.
test:palette	<code>node scripts/palette-audit.mjs</code>	Checks selected CSS text colors against 4.5:1 contrast.

12.2 Safe first-run and normal startup

The manifest requires Node $\geq 22.13.0$; the tracked runbook recommends Node 24. Use a runtime supporting the built-in SQLite and loader features. Choose the lockfile installation (`npm ci`) for a fresh checkout; avoid treating checked-in `node_modules` binaries as portable across machines.

First-time setup, only if the real files/database are absent:

```
npm ci
cp .env.example .env
cp server/.env.example server/.env
npm run db:migrate
npm run db:seed
```

Before using a development service beyond your private machine, set a unique `AUTH_SECRET` privately. Do not copy example files over existing configured secrets. For an existing database/version transition, take an online backup before schema changes and verify compatibility; do not blindly rerun historical upgrade instructions.

Normal sessions require two terminals in the repository root:

```
# Terminal 1
npm run api

# Terminal 2
npm run dev -- --force
```

Open `http://localhost:4173` to match the configured development auth origin. The API listens separately on 3001; Vite proxies `/api`. Stopping either process changes what is available. Running only Vite can display assets but backend mode gates initial session restoration and data reads on API availability.

Browser-only exploration uses root `VITE_DATA_MODE=demo` and a Vite restart. It has no real accounts or shared durable reports. Never change production flags just to bypass a missing configured server.

12.3 Build and hosted startup

Build: TypeScript validates browser/config -> Vite follows imports -> compiles JSX/TS, bundles translations/icons/components -> hashes CSS/JS under dist/assets -> copies public files -> emits dist/index.html. Build caches may change. HTML is an application shell, not server-rendered clinic pages.

Production release additionally asserts public admin exclusion. `npm run api` on the host still runs server source using `tsx`; `SERVE_STATIC=true` makes that process serve built browser files and `/api` from the same origin. `HOST/PORT` must suit the host; `DATABASE_PATH` must be permanent storage. Exact `APP_ORIGIN`, independent `AUTH_SECRET` and configured Resend values are required by deployed config. Templates use Node 24 and explicitly disabled automatic deployment.

Migrations run at runtime, not as a browser build step. The browser artifact contains `VITE_` settings, never server secret values. A domain, persistent disk, deployed service, email sender, backups and monitoring are external setup. The documentation task does not perform that transition.

12.4 Verification meaning and limits

Translation and palette audits test specific contracts; they are not professional language/accessibility certifications. Smoke tests cover built route/asset and shared demo logic; backend tests cover handler/auth/database semantics without external inbox delivery; browser tests cover the test API and UI in Chromium, not every Safari version or real hosting region.

Existing `verification.md` records historic checks. Earlier in this October 8 conversation the restored 2.1.4 typecheck, translation/palette checks, build, smoke, six backend tests and seven browser tests passed. This onboarding task does not rerun or claim new application tests; its new checks concern document completeness, secret exclusion, links, identical content, unchanged existing files and PDF layout. A prior pass does not verify cloud availability, real Resend delivery, live clinic data or current dependency security.

13. Database tables, keys and API reference

13.1 Every current application table

The current migration creates 24 tables; `database.ts` adds migrations bookkeeping, for 25 tables in total. A column ending `_id` generally links a related row. Foreign keys and transactions provide actual data constraints; TypeScript interfaces alone cannot protect stored relationships.

Table	Purpose and relationships
migrations	Created in <code>database.ts</code> ; applied SQL filename and timestamp prevent repeating migration files.
user	Better Auth identities, normalized unique email, verification state, names and timestamps. Parent for private ownership.
session	DB-backed login session/token/expiry, belongs to user with deletion cascade. Raw IP/user-agent deliberately cleared by auth hook.
account	Authentication provider identity/password credential data; belongs to user. Password storage is handled by Better Auth, not plaintext app code.
verification	Hashed OTP/provider verification values with identifiers and expiry; cleanup/scoped deletion independent of a user foreign key.

Table	Purpose and relationships
environment	Single marker row prevents the same DB from being opened for a different environment.
profiles	One private user profile: sex, gender, kg/cm measurements, age, language, notification preference; delete cascades.
account_metadata	Verified-at metadata used for signup event deduplication and account directory.
administrators	Trusted user role grant timestamp; no signup role input.
clinics	Rich clinic JSON document plus update timestamp. Referenced by most care data.
providers	Provider JSON tied to clinic; rebuilt by writeClinic.
clinic_languages	Clinic/spoken-language composite key. Distinct from interface locale.
clinic_insurance	Clinic/insurance composite key.
source_metadata	Clinic source, fictional flag and refresh time; used to retain provenance.
saved_clinics	User/clinic composite key for private favorites; cascade from user and clinic.
visit_plans	User/plan ID composite key, referenced clinic, plan JSON and creation timestamp. Owner-scoped ID prevents another user overwriting it.
reports	Clinic, optional user, pseudonymous actor/key/fingerprint, kind/mode, total or elapsed, submit/date, review status and sanitized public JSON. User deletion removes owned reports.
estimates	Current computed result by clinic/mode with timestamp.
estimate_history	Hourly result JSON by clinic/mode/hour; retained 90 days by jobs. Not currently the browser historical chart source.
daily_metrics	UTC day/metric aggregate counts with no individual ownership link.
presence	Recent pseudonymous guest or account-session activity, with user/session references and last_seen.
rate_limits	Persistent bucket count and reset time; API/auth/mail/report defenses survive restart.
launch_tasks	Twelve operator setup tasks, status, notes, explanation/action/verification and guide pointer.
audit_log	Admin mutation action/target/changes/timestamp with actor ID; actor deletion sets NULL.
job_runs	Job timestamps/status/attempts/error code for estimates, retention and directory refresh.

JSON-validity checks guard clinic/plan/report document storage. Report CHECK constraints distinguish completed positive totals from ongoing elapsed measurements. Saved-plan ownership uses a composite key, not one global plan ID. Auth indices, report evidence/owner indices and verification identifier indices support routine lookups.

13.2 Public and private API endpoints

Every path is prefixed /api. JSON writes must include the exact configured Origin and application/json content type. These are real HTTP operations, not React routes. Unless specified, GETs return JSON. Unknown paths/method combinations fall through to explicit errors.

Method and path	Behavior / access
GET /health	DB/schema/environment/mail/basic latest-job readiness; not proof provider delivery works.
GET /me	Guest null identity or current verified owner's private profile, saves/plans and admin flag.
GET /clinics?filters=...	Strict JSON search filters; directory with current evidence and accepted public reports.
GET /clinics/:id	One clinic view; 404 when absent, service error when operation fails.
POST /reports	Guest or account operational report; Idempotency-Key mandatory, validation/deduplication/review/limits.
POST /presence	Visible/active session heartbeat storage; may set signed guest cookie.
GET /integrations/ratings/:id	Configured rating adapter result or unavailable.
GET /integrations/travel/:id	Configured travel adapter result or unavailable.
GET /integrations/booking/:id	Configured availability result or unavailable, not a reservation operation.
PUT /profile	Verified owner profile/name update.
PUT /preferences	Verified owner interface language update.
PUT /saved-clinics/:id	Verified owner saves an existing clinic.
DELETE /saved-clinics/:id	Verified owner removes a saved clinic.
PUT /plans	Verified owner inserts/updates strict plan for supported clinic/mode.
GET /plans/:id	Reads a plan only within verified owner.
DELETE /plans/:id	Deletes a plan only within verified owner.
DELETE /account	Verified owner plus current password and DELETE confirmation; cascades and recalculates.

An X-MediQ-Account header can detect that the browser's expected account differs from the session, returning ACCOUNT_CHANGED. It does not select ownership or authorize a user. All private owner IDs come from verified Better Auth session state.

13.3 Allowlisted authentication endpoints

Auth path	Normal use
POST /auth/sign-up/email	Validated signup with first/last name, email, password; pending identity and verification send.
POST /auth/sign-in/email	Verified email/password login and cookie session.
POST /auth/sign-out	Current session logout.
GET /auth/get-session	Better Auth current-session read.
POST /auth/email-otp/send-verification-otp	Resend email-verification code only; rejects unsupported OTP purpose.
POST /auth/email-otp/verify-email	Verify six-digit code.
POST /auth/email-otp/request-password-reset	Request recovery code.
POST /auth/email-otp/reset-password	Reset password with code.
POST /auth/change-password	Current/new password; server forces other-session revocation.

These are the application's allowlisted provider paths, not every endpoint Better Auth could support. Raw bearer token fields are removed before responses reach browser application code. HTTP cookies still carry session behavior.

13.4 Internal endpoints

All internal endpoints require a verified user, administrators role and enabled server environment. Production admin access is disabled independently of the frontend.

Method and path	Behavior
GET /admin/guide	Returns docs/runbook.md as protected plain text.
GET /admin/accounts?q=...&page=...	Verified account names/emails/timestamps/status, escaped search and 20-row pagination. No optional profile fields.
GET /admin/metrics	Aggregate counts, five-minute presence, seven-UTC-day recent totals, latest job statuses; bookings null.
GET /admin/checklist	Persistent launch task records.
PUT /admin/checklist/:id	Status/notes update with audit record.
GET /admin/reports	Up to 100 reports currently needing review.
PUT /admin/reports/:id	Accept/reject existing review report with reason; audit/counters/recalculation.
PATCH /admin/clinics/:id	Restricted allowed directory corrections, provenance preserved and audit recorded.
GET /admin/audit	Up to 100 recent mutation records.

Important error categories include SIGN_IN_REQUIRED, ACCOUNT_CHANGED, ADMIN_REQUIRED, ORIGIN_DENIED, JSON_REQUIRED, INVALID_INPUT, INVALID_REPORT, INVALID_PLAN, IDEMPOTENCY_REQUIRED, RATE_LIMITED, NOT_FOUND, ALREADY_REVIEWED and SERVICE_UNAVAILABLE. `api.errorKey` maps several to user-readable translated messages; `status/code` matter more than one raw error string.

14. Working safely in this repository and finding the right file

14.1 Where to start for common changes

Desired change	Primary files, then connected checks
Add a new screen/URL	<code>src/pages</code> , <code>App.tsx</code> route/title map, <code>navigation/Layout</code> as needed; route/browser tests and translations
Change clinic search behavior	<code>services/search.ts</code> , filter contracts, server filters schema, <code>FindCarePage</code> ; <code>search/backend/browser</code> tests
Change report fields	<code>types.ts</code> , <code>utils/report.ts</code> , <code>QuickReport/ReportPage</code> , <code>server/validation.ts</code> , <code>application.ts</code> , schema if stored; <code>report/privacy</code> tests
Change estimate rules	<code>server/estimator.ts</code> , <code>jobs.ts</code> , application <code>clinicView</code> , confidence UI; estimator and report tests
Change login/verification	<code>AccountPage</code> , <code>services/api</code> , <code>server/application/auth</code> ; OTP/lifecycle/browser tests
Change private settings	<code>account.ts</code> , <code>SettingsPage</code> , <code>profileSchema</code> , <code>/profile</code> handler and schema; owner isolation/deletion tests
Add authorized data provider	<code>server/integrations.ts</code> registry/contracts, jobs refresh or integration endpoints; do not put keys in <code>VITE_</code> values
Change brand/layout	<code>styles.css</code> , <code>Brand.tsx</code> , <code>public/favicon.svg</code> and shared components; <code>palette/responsive</code> checks
Add/modify language	<code>translate</code> , <code>expandedTranslations</code> , named catalogs, data language choices and server locale schema; i18n/RTL tests
Prepare public hosting	deploy templates, build-release, server config/index and runbook; real provider setup remains separate

14.2 Important maintenance boundaries

Work on source, not generated `dist/node_modules/cache` files. Preserve configured real env files and personal data. A clean main branch and reproducible install/build are useful starting points, but do not remove historically tracked dependencies as part of an unrelated change. Such cleanup would require its own scoped change and verification.

Do not alter a migration already applied to a real database to pretend history changed. Add a new migration for future schema changes and back up first. Rolling source back across incompatible schemas needs a matching backup or carefully designed compatible migration. Different environments use independent databases and secrets; a copied development DB is not a safe production bootstrap.

The current root manifest has no lint script, no dedicated tests/healthcare.test.ts, no source accessibility/measurement helper modules from 2.1.5, and only migration 001. Documentation referring to later files must not be interpreted as instructions to invent them into 2.1.4. Current source shape takes precedence over historical product notes.

A static rewrite fixes deep links; it does not run the API. Preview on its own is not full-stack deployment. Neither hidden navigation nor a staging branch makes a website private. The public demo transition should address stable fictional data, real email if accounts are offered, actual access separation, backups and external availability without exposing developer internals.

14.3 Troubleshooting by symptom

- Initial session screen never becomes ready: confirm API runs, /api proxy/origin/config align and SQLite schema matches current source. Do not mask it by switching adapters silently.
- Signup says code sent but inbox is empty locally: MAIL_MODE=file writes to the configured private mail directory; it is intentionally not external email delivery.
- Saves disappear after reload: guest personal state is memory-only. Verify/sign in to get SQL-backed private persistence.
- One completed report does not produce a live estimate: three eligible completed samples and fresh evidence are required; an ongoing report never fills that threshold.
- Deep-link refresh shows a host 404: host must serve index.html for browser routes; API endpoints require real server routing.
- Old UI appears in preview after a version change: rebuild dist from current source and refresh Vite cache when developing; tracked bundles can be historical.
- Admin link missing: check browser environment, verified role and server environment. Production deliberately has no internal UI/API.
- Backend report note is missing from feed: notes and exact input timestamps are intentionally not retained in the backend public document.

14.4 Documentation authority and delivery

This guide was built from current tracked source plus a privacy-preserving local file inventory. It groups external/generated content explicitly rather than suggesting each dependency is authored by MediQ. Existing docs contain historical assumptions; particularly product-background.md and the ignored personal setup guide require context.

The two deliverables are docs/onboarding/README.md and docs/onboarding/MediQ-2.1.4-onboarding.pdf. They are documentation, not runtime imports. The PDF contains the same sections/catalogs/coverage as the README, with pagination, table of contents and bookmarks. No source, dependency, env, database or root README edit is required for these deliverables.

Appendix A. Exact tracked-file coverage and grouped exclusions

The source snapshot contains 4,741 tracked paths: 4,643 installed dependency paths, five generated dist files and 93 other tracked files. Every one of the 93 other paths has an individual description in this guide. The real environment files and local-only folder families are covered separately even though they are not tracked. The documents being delivered are new additions after that source snapshot.

The following checklist is an explicit complete inventory of the individually covered original tracked files, not another copy of their explanations:

- .DS_Store
- .env.example
- .env.production.example
- .env.staging.example
- .gitignore
- README.md
- deploy/render.production.yaml
- deploy/render.staging.yaml
- docs/backend-backlog.md
- docs/data-and-estimates.md
- docs/next-version-implementation-prompt.md
- docs/product-background.md
- docs/runbook.md
- docs/upgrade-checklist.md
- docs/verification.md
- index.html
- package-lock.json
- package.json
- playwright.config.ts
- public/_redirects
- public/favicon.svg
- scripts/build-release.mjs
- scripts/e2e-server.ts
- scripts/il8n-audit.mjs
- scripts/palette-audit.mjs
- scripts/serve-dist.mjs
- scripts/smoke-test.mjs
- server/.env.example
- server/.env.production.example

- server/.env.staging.example
- server/application.ts
- server/auth.ts
- server/cli.ts
- server/config.ts
- server/database.ts
- server/estimator.ts
- server/index.ts
- server/integrations.ts
- server/jobs.ts
- server/migrations/001_initial.sql
- server/security.ts
- server/seed.ts
- server/validation.ts
- src/App.tsx
- src/components/Brand.tsx
- src/components/ClinicComponents.tsx
- src/components/Layout.tsx
- src/components/ProductTour.tsx
- src/components/QuickReport.tsx
- src/components/UI.tsx
- src/context/AppContext.tsx
- src/context/expandedTranslations.ts
- src/context/interfaceTranslations.ts
- src/context/locales/ar.json
- src/context/locales/es.json
- src/context/locales/gu.json
- src/context/locales/hi.json
- src/context/locales/pl.json
- src/context/locales/zh.json
- src/context/translate.ts
- src/data/mockData.ts
- src/main.tsx
- src/pages/AccountPage.tsx
- src/pages/AdminPage.tsx
- src/pages/ClinicPage.tsx
- src/pages/ContactPage.tsx
- src/pages/FindCarePage.tsx

- src/pages/KnowPage.tsx
- src/pages/OnboardingPage.tsx
- src/pages/ReportPage.tsx
- src/pages/SavedPage.tsx
- src/pages/SettingsPage.tsx
- src/services/account.ts
- src/services/api.ts
- src/services/clinicService.ts
- src/services/locationService.ts
- src/services/search.ts
- src/styles.css
- src/types.ts
- src/utils/navigation.tsx
- src/utils/report.ts
- src/utils/time.ts
- src/vite-env.d.ts
- tests/backend.test.ts
- tests/browser/mediq.spec.ts
- tsconfig.app.json
- tsconfig.app.tsbuildinfo
- tsconfig.json
- tsconfig.node.json
- tsconfig.node.tsbuildinfo
- tsconfig.server.json
- vercel.json
- vite.config.ts

Tracked generated distribution files

These five are covered as generated build output rather than decoded/minified implementation:

- dist/_redirects
- dist/assets/index-C8to0N5o.css
- dist/assets/index-CRTJQIoz.js
- dist/favicon.svg
- dist/index.html

Tracked external package coverage

All 4,643 tracked node_modules paths are grouped by installed package family in chapter 9. This table gives exact tracked counts under each first package namespace, including hidden folders. Scoped-package children are listed in chapter 9. No package code, license text, sourcemap data or declaration line is individually repeated.

Dependency root	Tracked paths
node_modules/.bin	5
node_modules/.package-lock.json	1
node_modules/.vite	15
node_modules/@oxc-project	4
node_modules/@rolldown	11
node_modules/@types	33
node_modules/@vitejs	8
node_modules/csstype	5
node_modules/detect-libc	8
node_modules/fdir	7
node_modules/fsevents	6
node_modules/lightningcss	14
node_modules/lightningcss-darwin-arm64	4
node_modules/lucide-react	4090
node_modules/nanoid	25
node_modules/picocolors	7
node_modules/picomatch	10
node_modules/postcss	55
node_modules/react	27
node_modules/react-dom	43
node_modules/rolldown	54
node_modules/scheduler	15
node_modules/source-map-js	18
node_modules/tinyglobby	10
node_modules/typescript	132
node_modules/vite	36

Explicit omissions and why

- Real secret values, account credentials, cookies/tokens, OTPs and private record contents: excluded because onboarding requires configuration roles/schema, not disclosure.
- Individual dependency source/type/license/binary lines: grouped because third-party internals are outside authored application scope and the reader requested purpose/function rather than thousands of third-party lines.
- Individual package-lock entries, minified bundles, sourcemaps, compiler cache records and Finder binary metadata: grouped because they are generated/derived. Their purpose, location and relationship are documented.
- Individual compressed .git object contents and historical file versions: grouped because this is current-main onboarding, not a full historical code review. Refs/logs/index/hooks/objects are explained.
- Individual test-mail UUID names and message contents: grouped as private generated mailbox artifacts. Counts and transport/schema roles are sufficient.
- Local database rows and binary pages: schema/file lifecycle described; no people's data is exported.
- Files outside this repository (global tool caches, OS keychains, hosting dashboards, another checkout): not repository content. No cloud inspection is implied.

No first-party source/config/script/test/document file in the original tracked snapshot was skipped. No currently absent 2.1.5 source is presented as part of 2.1.4.

Appendix B. Folder coverage ledger

The ledger lists every first-party/local operational folder observed, excluding repetitive external package internals and Git hash buckets already grouped above. Directory counts are snapshot facts, not immutable specifications.

- .data/
- .data/archives/
- .data/archives/2026-10-08-v2.1.5/
- .data/development/
- .data/development/mail/
- .data/e2e/
- .data/e2e/mail/
- .git/
- .git/hooks/
- .git/info/
- .git/logs/
- .git/logs/refs/
- .git/logs/refs/heads/
- .git/logs/refs/heads/codex/
- .git/logs/refs/remotes/
- .git/logs/refs/remotes/origin/
- .git/objects/

- .git/refs/
- .git/refs/heads/
- .git/refs/heads/codex/
- .git/refs/remotes/
- .git/refs/remotes/origin/
- .git/refs/tags/
- deploy/
- dist/
- dist/assets/
- docs/
- docs/onboarding/
- node_modules/
- playwright-report/
- public/
- scripts/
- server/
- server/migrations/
- src/
- src/components/
- src/context/
- src/context/locales/
- src/data/
- src/pages/
- src/services/
- src/utils/
- test-results/
- tests/
- tests/browser/

The `node_modules/` family includes each package root/scoped child described in chapter 9; its nested implementation folders inherit that grouped explanation. `.git/objects/` includes hash-prefix buckets, pack and info as described in chapter 10. Empty folders are local operational scaffolding, not necessarily tracked objects; Git normally tracks files rather than empty directories.

Appendix C. Glossary and source navigation

Term	Meaning in MediQ
SPA	Single-page application; browser React changes pages without replacing the HTML document for ordinary links.
API	Server HTTP operations used for accounts, directory, reports and private saves.
Origin	Scheme, hostname and port identifying the browser site; write/auth checks require the configured origin.
Adapter	Implementation behind a shared interface, such as mock vs backend clinic service or a future vendor connection.
Cookie session	Browser sends an HttpOnly cookie; server checks the stored authenticated session.
OTP	Short-lived one-time verification/recovery code, hashed in auth storage; local development delivery is a private file.
HMAC	Keyed hash used to sign/bucket pseudonymous guest/email/network identities.
Idempotency	Retrying one logical write with the same key returns the prior result instead of duplicating it.
Transaction	Related SQL changes commit together or roll back together.
WAL	SQLite write-ahead log; part of consistent live-database state.
Migration	Numbered schema operation applied once and recorded.
Fixture/seed	Deliberately fictional source data / command loading permitted fixtures into a database.
Staging	Separate test release environment; privacy needs access protection as well as separate settings.
Production	Public release configuration with internal admin disabled; not automatically proof that data is real.
Build artifact	Generated dist output and assets that a host serves.
Lockfile	Dependency resolution record used for reproducible installation.

Start with `src/main.tsx` and `src/App.tsx` for the browser, `server/index.ts` and `server/application.ts` for the server, `server/migrations/001_initial.sql` for storage, and `docs/runbook.md` for operation. Use the file catalogs and endpoint tables to follow the specific feature you are changing. This guide describes source commit 73f5f74; later modifications should update both deliverables together.